

Software Requirements Specification for

CampusNexus

A Campus-Centric Rental & Community Services Platform

CSE 3210: Software Project LAB (SPL)

B.Sc. in Computer Science and Engineering

SUBMITTED BY

Name	ID	Batch / Year
Abdullah Al Noman	12208004	21-22 / 3th Year

SUPERVISED BY

Md. Zahidur Rahman

Lecturer,

Department of CSE, Comilla University

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COMILLA UNIVERSITY**

Submission Date: 10 May 2026

Chapter 1

1. Introduction

1.1 Purpose

This document defines the software requirements for CampusNexus, a web-based rental and community service platform designed for university students. CampusNexus facilitates renting physical items such as books, electronics, and sports gear, as well as skill-based services such as academic tutoring, assignment help, editing, photography, and extracurricular coaching.

Additionally, the system provides a News & Events feature, enabling students and campus communities to stay updated on university happenings. CampusNexus places a strong emphasis on robust database design and operations, covering complex queries, relational integrity, status tracking, analytics, and multi-table interactions.

The mission of CampusNexus is to:

- Simplify access to goods and services for students.
- Promote peer-to-peer skill sharing and collaboration.
- Foster a connected and resourceful campus environment.
- Extend the idea of rental to include time, skills, and experiences beyond physical items.
- Demonstrate advanced database design and operations as the core technical foundation.

1.2 Intended Audience

The primary stakeholders include:

- Administrators - Manage users, monitor transactions, enforce rules.
- Developers - Implement platform features as per specifications.
- QA/QC Engineers - Test and validate functionality, usability, and performance.
- End Users (Renters, Service Providers, Skill Seekers, Event Followers).
- Course Supervisor - Evaluate database design, operations, and implementation quality.

1.3 Intended Use

Each stakeholder group uses this SRS differently:

- Administrators: Understand backend functionalities, user management, and moderation tasks.
- Developers: Implement features, database schema, REST APIs, and user interfaces.
- QA/QC Engineers: Design test cases based on functional and non-functional requirements.
- Course Supervisor: Evaluate CLO fulfillment, database complexity, and implementation quality.
- End Users: Understand available features and expected system behavior.

1.4 Product Scope

CampusNexus is a campus-focused web application where students can rent physical items and offer or hire skill-based services. The platform is built as a full-stack web application using Django (backend), React (frontend), and MySQL (database). The database layer is the core focus, featuring 10+ relational tables, complex JOIN queries, aggregate statistics, status tracking workflows, and a notification system — all implemented purely through database operations without third-party complexity.

1.4.1 Benefits and Objectives

- **Helping Students Earn:** Students earn by renting items or offering services such as tutoring or photography.
- **Making Renting Simple:** Students find needed items or skills without permanent ownership cost.
- **Building Trust:** Ratings and reviews create accountability between users.
- **Strengthening Campus Community:** Events, skill sharing, and peer rentals encourage collaboration.
- **Demonstrating Database Mastery:** All core features are driven by rich SQL operations — JOINS, aggregations, transactions, and status workflows.

1.5 Risk Definition

Potential risks in the operation of CampusNexus:

- **User Inactivity:** Students may not actively list items or offer services, limiting platform value.
- **Trust Issues:** Users may doubt the quality of listed skills or the condition of rental items.
- **Security Vulnerabilities:** User data or sessions may be exposed to unauthorized access.
- **Data Integrity Issues:** Improper database transactions may lead to inconsistent rental or payment records.
- **Scope Creep:** Adding complex features like real-time chat or payment gateways beyond project timeline.

Chapter 2 – Overall Description

2.1 User Classes and Characteristics

CampusNexus serves three distinct user classes:

2.1.1 Borrower (Renter)

- Primary Goal: Temporarily access needed items or skills cost-effectively.
- Activities: Browse listings, add to cart, place rental orders, track rental status, submit reviews.
- Responsibilities: Return items on time and in good condition, adhere to rental agreements.

2.1.2 Lender / Provider (Rentee)

- Primary Goal: Earn income by monetizing underutilized assets or skills.
- Activities: Create and manage listings, approve/reject rental requests, update availability calendar.
- Responsibilities: Accurate item descriptions, timely responses to booking requests.

2.1.3 Administrator

- Primary Goal: Ensure the platform operates securely and efficiently.
- Activities: Verify users, moderate listings and events, view analytics dashboard, resolve disputes.
- Responsibilities: Uphold community guidelines, protect user data, maintain platform integrity.

2.2 User Needs

2.2.1 Renters

- Search items by category, price range, condition, and availability.
- View item details with availability calendar and owner reviews.
- Track rental status from request to completion.
- Submit ratings and reviews after rental completion.
- Receive database-driven notifications for rental updates.

2.2.2 Lenders / Providers

- User-friendly listing interface with images, pricing, deposit, and category.
- Approve or reject incoming rental requests.
- View rental history, earnings summary, and item availability.
- Build credibility through accumulated ratings and reviews.

2.2.3 Skill Providers

- Create service listings with hourly/daily rates and availability schedule.
- Manage bookings and track service history through the dashboard.
- Receive ratings and reviews to strengthen reputation.

2.2.4 Skill Seekers

- Search and filter service providers by skill, price, and rating.
- Book services and track booking status.

- View provider profiles with reviews and qualifications.

2.2.5 Event and News Followers

- Browse a centralized campus events and news feed.
- Filter events by category and date.
- Receive database-driven notifications for upcoming events.

2.2.6 Administrators

- Analytics dashboard showing platform-wide statistics.
- Tools to verify listings, manage users, and approve events.
- View and resolve dispute and refund requests.
- Monitor transaction history and generate reports.

2.3 Operating Environment

2.3.1 Hardware Platform

- Client-side: Any modern web browser on desktop or mobile device.
- Server-side: Cloud-hosted Linux environment (Railway / Render for backend, Vercel for frontend).

2.3.2 Software Components

Layer	Technology	Purpose
Frontend	React.js + JavaScript	Component-based SPA user interface
Styling	CSS3 + Tailwind CSS	Responsive, mobile-friendly design
Backend	Django + Django REST Framework	REST API, business logic, admin panel
Authentication	JWT (simplejwt)	Secure stateless session management
Database	MySQL 8.0+	Primary data store — core project focus
DB Connector	mysqlclient (Python)	Django-MySQL connection driver
HTTP Client	Axios	React to Django API communication
Notifications	Database-driven (MySQL)	In-app alerts without third-party services
Hosting (FE)	Vercel	React frontend deployment (free tier)
Hosting (BE)	Railway / Render	Django backend deployment (free tier)

2.3.3 Network Requirements

- Requires stable internet connection for full functionality.
- All client-server communication over HTTPS.
- Notification polling handled via periodic API requests (no WebSocket needed).

2.3.4 Security Considerations

- All communication encrypted via HTTPS/TLS.
- Passwords hashed using Django's built-in PBKDF2 algorithm.

- JWT tokens for stateless, secure authentication.
- Input validation and SQL injection prevention via Django ORM.
- Role-based access control (Renter, Lender, Admin).

2.4 Constraints

- Web-only platform during initial launch; fully responsive for mobile browsers.
- No real-time WebSocket features — notifications handled via database polling.
- No third-party payment gateway — simulated payment flow for demonstration.
- Development team limited to one (1) member.
- Must comply with university data privacy and security policies.
- Free-tier cloud services only (Vercel, Railway, PlanetScale).

2.5 Assumptions

- Users are familiar with web applications and basic digital transactions.
- University email addresses are available for registration verification.
- Stable internet connection available for all users.
- Administrator has sufficient authority to enforce platform rules.

Chapter 3 – Requirements

3.1 Functional Requirements

3.1.1 User Registration & Login

As a new user, I want to sign up using my university email and password so that I can create a verified account and access CampusNexus securely.

Success:

- User provides full name, university email, gender, and password → System creates and activates account.
- System verifies university email format → Account is activated and user can log in.
- User logs in with valid credentials → JWT token issued; session begins.

Failure:

- Required fields missing → System shows field-specific error messages.
- Duplicate email → System shows: "This email is already registered. Please log in."
- Invalid credentials on login → System shows: "Invalid email or password."

3.1.2 Item & Skill Listing (Post)

As a lender or skill provider, I want to create listings for items or services so that other students can find and rent them.

Success:

- User fills listing form (title, description, category, price, deposit, condition, images) → System publishes listing.
- User updates or deletes an existing listing → System reflects changes immediately.

Failure:

- Required fields missing → System highlights missing fields and prevents submission.
- Invalid price or deposit value → System shows validation error.

3.1.3 Smart Availability Calendar

As a renter, I want to see a calendar showing when an item is available or booked so that I can plan my rental dates accurately.

Success:

- Item detail page shows a monthly calendar with available (green) and booked (red) dates.
- Calendar data is generated from the Rental table using date range queries.
- Renter selects available start and end dates → System calculates total cost automatically.

Failure:

- Selected dates overlap with existing rental → System blocks submission and shows conflict message.

3.1.4 Search & Filter

As a user, I want to search and filter listings so that I can quickly find the exact item or service I need.

Success:

- User enters keyword → System returns matching listings via full-text search.

- User applies filters (category, price range, condition, rating) → System returns filtered results.
- User sorts by price, rating, or newest → System reorders results accordingly.

Failure:

- Empty search query → System prompts user to enter a keyword.
- No results found → System shows: "No items found. Try a different search."

3.1.5 Cart & Checkout

As a renter, I want to add items to my cart and complete checkout so that I can confirm my rental easily.

Success:

- User adds item to cart with rental dates → System stores cart entry in database.
- User proceeds to checkout → System shows order summary with total cost and deposit.
- User confirms order → System creates Rental and Transaction records; item marked as booked.

Failure:

- Item already booked for selected dates → System blocks checkout and shows conflict.
- Cart is empty → System disables checkout button.

3.1.6 Rental Status Tracking

As a renter or lender, I want to track the status of every rental so that I always know the current state of a transaction.

Status Workflow:

- Requested → Approved → Active → Returned → Completed
- Owner can approve or reject a Requested rental.
- System automatically updates status to Active on rental start date.
- Owner confirms return → Status changes to Returned then Completed.
- Deposit is released back to renter upon Completed status.

Failure:

- Owner rejects request → Renter notified; status set to Rejected.
- Item not returned on time → System flags rental as Overdue.

3.1.7 Rating & Review System

As a renter, I want to rate and review an owner after rental completion so that other users can make informed decisions.

Success:

- Rental reaches Completed status → Renter can submit 1-5 star rating with written review.
- Owner average rating recalculated from all reviews using database aggregate query.
- Reviews displayed on item and user profile pages with timestamp.

Failure:

- Rental not completed → Rating submission blocked.
- Duplicate review → System shows: "You have already reviewed this rental."

3.1.8 User Dashboard with Statistics

As a user, I want a personal dashboard showing my activity statistics so that I can track my rentals, earnings, and platform engagement.

Renter Dashboard shows:

- Total rentals made, active rentals, completed rentals, pending requests.
- Total amount spent with monthly breakdown chart.
- Rental history table with status and dates.

Lender Dashboard shows:

- Total listings, total rentals received, total earnings.
- Most rented item, average rating received.
- Earnings chart by month using database aggregate queries.

Admin Dashboard shows:

- Platform-wide user count, listing count, total rentals, total transactions.
- Category-wise rental distribution chart.
- Recent reports and pending refund requests.

3.1.9 Database-Driven Notification System

As a user, I want to receive in-app notifications for important events so that I stay updated without missing anything.

Notification triggers:

- Rental request received (for owner).
- Rental approved or rejected (for renter).
- Rental status changed to Active, Returned, or Completed.
- New review received on a listing.
- Refund request status updated.
- New campus event posted.

Implementation:

- Notifications stored in MySQL Notification table with user_id, message, is_read, created_at.
- Frontend polls the notification API every 30 seconds for new entries.
- Unread count shown as badge on notification bell icon.

3.1.10 Refund Request

As a renter, I want to request a refund for a defective or undelivered item so that I can recover my payment.

Success:

- User navigates to rental history → selects order → submits refund request with reason and proof.
- System creates Refund record; admin and owner notified.
- Admin reviews and approves or rejects refund → User notified; transaction reversed if approved.

Failure:

- Required fields missing → System highlights errors.
- Refund policy conditions not met → System rejects with explanation.

3.1.11 Report System

As a user, I want to report a listing, service, or user so that inappropriate or fraudulent content is reviewed by admins.

Success:

- User selects Report on listing or profile → chooses reason category → submits details.
- System creates Report record; admin moderation queue updated.
- Admin reviews report → takes action (warning, suspension, removal).

Failure:

- Reason field empty → System blocks submission.
- Duplicate report → System shows: "You have already reported this."

3.1.12 News & Events

As a student, I want to browse campus news and events so that I stay informed about university activities.

Success:

- Admin posts verified events with title, description, date, category, and image.
- Students browse events filtered by category or date.
- Students register interest in an event → stored in EventRegistration table.
- Notification sent to all users when a new verified event is posted.

Failure:

- Unverified event → Not shown on public feed until admin approves.

3.2 Non-Functional Requirements

3.2.1 Performance

- System responds to user actions within 2 seconds under normal load.
- Database queries optimized with indexes on foreign keys and frequently filtered columns.
- Platform handles 20% user surge during exam season without degradation.

3.2.2 Security

- All communication encrypted via HTTPS/TLS.
- JWT tokens expire after 24 hours; refresh tokens used for session extension.
- Django ORM prevents SQL injection; all inputs validated server-side.
- Role-based access control enforced on all API endpoints.

3.2.3 Reliability

- System maintains 99.9% uptime.
- Database transactions use ACID properties to prevent data corruption.
- Regular automated backups of MySQL database.

3.2.4 Usability

- Responsive design works on desktop and mobile browsers.
- Consistent UI with clear navigation, loading states, and error messages.
- At least 90% of surveyed users report satisfaction with ease of use.

3.2.5 Maintainability

- Modular Django app structure (users, items, rentals, events, notifications).
- RESTful API design for clean separation between frontend and backend.
- Code documented with inline comments and README files.

Chapter 4 – Database Design

The database is the core focus of CampusNexus. MySQL is used as the primary relational database. The schema is designed to support complex queries, relational integrity, status workflows, and aggregate analytics — directly addressing CLO 7 (Design, create, and process a database).

4.1 Database Tables Overview

#	Table Name	Purpose
1	User	Stores all registered users with role (renter/lender/admin)
2	Item	Stores rental/sell/skill listings with category, price, deposit
3	Rental	Records every rental transaction with status workflow
4	Transaction	Tracks all financial movements (payment, deposit, refund)
5	Cart	Temporary storage for items before checkout
6	Review	Stores ratings and reviews after rental completion
7	Notification	Database-driven in-app alerts for all user actions
8	Refund	Tracks refund requests and their resolution status
9	Report	Stores user-submitted reports against listings or users
10	Event	Campus news and events posted by admin
11	EventRegistration	Tracks which users registered interest in an event

4.2 Key Table Schemas

User Table

Field	Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(150)	UNIQUE, NOT NULL	University email
password	VARCHAR(255)	NOT NULL	Hashed password
role	ENUM	NOT NULL	renter / lender / admin
is_verified	BOOLEAN	DEFAULT FALSE	Email verification status
created_at	DATETIME	DEFAULT NOW()	Registration timestamp

Item Table

Field	Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique item identifier
owner_id	INT	FK → User.id	Item owner reference
title	VARCHAR(200)	NOT NULL	Item title

description	TEXT		Detailed item description
category	VARCHAR(50)	NOT NULL	Electronics/Books/Skills etc.
price	DECIMAL(10,2)	NOT NULL	Daily/hourly rental price
deposit	DECIMAL(10,2)	DEFAULT 0	Refundable security deposit
condition	ENUM	NOT NULL	new / used / fair
status	ENUM	NOT NULL	available / rented / unavailable
created_at	DATETIME	DEFAULT NOW()	Listing creation timestamp

Rental Table

Field	Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique rental identifier
item_id	INT	FK → Item.id	Rented item reference
renter_id	INT	FK → User.id	Renter user reference
start_date	DATE	NOT NULL	Rental start date
end_date	DATE	NOT NULL	Rental end date
total_cost	DECIMAL(10,2)	NOT NULL	Calculated total rental cost
deposit_paid	DECIMAL(10,2)	DEFAULT 0	Deposit amount held
status	ENUM	NOT NULL	requested/approved/active/returned/completed/rejected/overdue
created_at	DATETIME	DEFAULT NOW()	Booking timestamp

Notification Table

Field	Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique notification identifier
user_id	INT	FK → User.id	Recipient user
message	VARCHAR(255)	NOT NULL	Notification message text
is_read	BOOLEAN	DEFAULT FALSE	Read/unread status
created_at	DATETIME	DEFAULT NOW()	Notification timestamp

4.3 Key Database Operations

The following database operations demonstrate the complexity and depth required by CLO 7:

Operation Type	Example Use Case
----------------	------------------

Multi-table JOIN	Get rental details with item name, renter name, and owner name in one query
Aggregate (SUM)	Calculate total earnings for a lender across all completed rentals
Aggregate (COUNT)	Count total rentals per category for admin analytics dashboard
Aggregate (AVG)	Calculate average rating for a user from all received reviews
Date Range Query	Find all rentals overlapping a requested date range for availability check
Status Update Transaction	Update rental status and create notification record atomically
Subquery	Find top-rated items in each category for homepage featured section
GROUP BY + ORDER BY	Monthly earnings report grouped by month, ordered by date
Conditional Filter	Search items by keyword, category, price range, and condition together
Cascade Delete	Remove all cart entries when a rental order is confirmed

Chapter 5 – Technology Stack

5.1 Overview

CampusNexus is a full-stack web application. The technology stack is chosen to match the developer's existing skills (Django, React, JavaScript, HTML, CSS, MySQL) while keeping the architecture simple and focused on database depth rather than third-party service complexity.

5.2 Frontend

Technology	Version	Purpose
React.js	18+	Component-based SPA — pages, forms, dashboard, listings
JavaScript (ES6+)	ES2022	Frontend logic, form handling, API calls
HTML5	5	Semantic page structure and markup
CSS3 + Tailwind CSS	Latest	Responsive styling and mobile-first layout
Axios	Latest	HTTP requests from React to Django REST API
React Router v6	6+	Client-side page navigation without reload
Recharts / Chart.js	Latest	Dashboard statistics and analytics charts

5.3 Backend

Technology	Version	Purpose
Django	4.2+	Core backend — models, views, admin panel, ORM
Django REST Framework	3.14+	Build JSON REST APIs consumed by React frontend
djangorestframework-simplejwt	Latest	JWT token generation and validation for auth
django-cors-headers	Latest	Allow React frontend to call Django API
Pillow	Latest	Image upload and processing for item photos

5.4 Database

Technology	Version	Purpose
MySQL	8.0+	PRIMARY DATABASE — all relational data, core project focus
mysqlclient	Latest	Python driver connecting Django to MySQL
Django ORM	Built-in	Model-based queries, migrations, and relationships

5.5 New Skills to Learn (Priority Order)

Skill	Priority	Why Needed
Django REST Framework	HIGH	Build JSON APIs so React can communicate with Django backend
JWT Authentication	HIGH	Secure stateless login — token issued on login, sent with every request

Axios + React Router	HIGH	API calls from React and multi-page navigation
Tailwind CSS	MEDIUM	Fast responsive UI without writing lots of custom CSS
Recharts / Chart.js	MEDIUM	Dashboard charts for statistics — pure frontend library
Django ORM advanced queries	MEDIUM	Complex JOINS, aggregations, and annotations in Python
Vercel + Railway deployment	LOW	Deploy React and Django on free cloud platforms

5.6 Recommended Learning Timeline

Week	Focus
Week 1	Django REST Framework — build User, Item, and Rental APIs
Week 2	JWT Authentication — secure all endpoints; React login/signup integration
Week 3	React frontend — homepage, listings, item detail, search and filter
Week 4	Cart, Checkout, Rental Status Tracking workflow
Week 5	Rating & Review system; Availability Calendar
Week 6	Dashboard with statistics charts; Notification system (database polling)
Week 7	News & Events; Refund & Report features; Admin panel
Week 8	Testing, bug fixing, UI polish, and deployment on Vercel + Railway

Chapter 6 – Syllabus CLO Fulfillment

The following table demonstrates how CampusNexus directly addresses each Course Learning Outcome (CLO) of CSE 3210: Software Project LAB (SPL).

CLO	Requirement	How CampusNexus Fulfills It
CLO 1	Client-server software	Django REST API (server) + React SPA (client) over HTTPS
CLO 2	Retrieve data from forms to produce displays	Listing forms, rental forms, search results, dashboard reports all pull from MySQL
CLO 3	Strings, dates, arrays, functions, classes, objects	Django models (classes), date range queries, array rendering in React, utility functions
CLO 4	User interfaces to collect and present data	React UI with forms, tables, charts, calendars, and notification bell
CLO 5	Validation, exceptions, file and database storage	Server-side validation in DRF serializers, exception handling, image file upload, MySQL storage
CLO 6	Secure website implementation	HTTPS, JWT auth, hashed passwords, role-based access, ORM SQL injection prevention
CLO 7	Design, create, and process a database	11-table MySQL schema, JOINS, aggregations, date queries, status workflows, transactions — CORE FOCUS
CLO 8	Pages for a typical web application	Home, Listings, Item Detail, Cart, Dashboard, Profile, Events, Admin — full web app

— End of Proposal: CampusNexus —